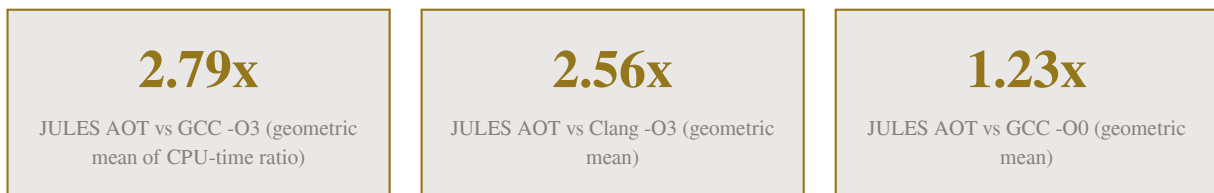


Table of Contents

1. Executive Summary	2
2. Benchmark Methodology	3
2.1 Environment and Compilers	3
2.2 Why CPU Time Is the Primary Metric	3
2.3 Measurement Protocol	3
3. Correctness Verification	4
4. Runtime Performance	4
5. Compile-Time Performance	6
6. Why the Gap: Assembly-Level Analysis	7
6.1 The Accumulator-Loop Call Protocol (fib)	7
6.2 Register-Pressure Rounding Error (mandel, inthash)	7
6.3 No Vectorization (primes vs Clang)	7
7. Conclusions and Recommendations	8

1. Executive Summary

This report benchmarks the JULES compiler against two production compilers, GCC 14.2.0 and LLVM/Clang 19.1.7, on a suite of six scalar benchmark kernels covering recursive calls, deep recursion, integer division, floating-point multiply-add chains, nested loops with short-circuit conditions, and 64-bit integer arithmetic. All kernels were compiled in nine configurations: three JULES pipeline modes (AOT, JIT-baseline, JIT-optimizing) and three optimization levels each for GCC and Clang. Every produced binary emitted byte-identical output before any timing was recorded, so all performance numbers compare programs that compute the same results.



The headline result is that JULES-compiled code runs 2.79 times slower than GCC -O3 and 2.56 times slower than Clang -O3 on the geometric mean, while sitting 1.23 times off unoptimized GCC. On fib and inthash JULES now beats GCC -O0 outright. This revision of the report follows a backend overhaul: a real register allocator (pass 85, linear scan over backwards-liveness-derived live ranges with callee-saved, caller-saved and XMM pools plus density-ranked, backedge-aware spill heuristics), frame-pointer elision for functions whose values all fit registers, fused compare-and-branch lowering, an FP constant pool with loop-invariant materialization hoisting, and copy-chain elimination in the machine peephole. Against the previous measurement the geometric-mean deficit shrank from 4.53x to 2.79x: primes reached exact GCC -O3 parity, inthash improved from 5.89x to 2.05x, mandel halved from 10.5x to 5.5x, and fib moved from 4.90x to 3.49x. The worst remaining case is mandel (5.5x), which is bound by FP instruction selection and the allocator's lack of live-range splitting; the best cases are primes (1.00x, divide-bound for both compilers) and inthash (2.05x).

Two findings deserve equal billing with the performance numbers. First, the benchmarks and the new optimization-level matrix caught four more real miscompiles on top of the two from the first measurement round: the initial register allocator computed live ranges as linear intervals, which is unsound when a loop backedge re-reads a value whose linear last-use precedes the latch; the inliner's resume-block repinning could strand pure nodes left behind at the call-site block (a use-before-def that only surfaces at -O0/-Og where no post-inline folding repairs the graph); SROA refused to promote allocations referenced by dead users or by memory phis, which left loop counters in heap cells; and the cast lowering read constant operands from stack slots that are never written. All six are fixed and locked in with regression tests, and every program is now verified byte-identical across all seven optimization levels and both JIT modes. Second, JULES compiles these kernels 2.4 times faster than GCC -O3 and 3.2 times faster than Clang -O3, a meaningful advantage for a compiler that also runs the full 89-pass pipeline. Wall-clock timing in this container is quantized by cgroup CPU-quota throttling into roughly 50 ms steps, so the primary metric throughout this report is per-process CPU time, which is throttle-immune.

2. Benchmark Methodology

2.1 Environment and Compilers

The benchmark ran on a two-core Intel Xeon container running Debian trixie, with GCC 14.2.0 from the system toolchain and LLVM/Clang 19.1.7 installed alongside it. Clang is the LLVM project's production frontend; benchmarking Clang at -O2/-O3 exercises the LLVM 19 optimization pipeline and x86-64 backend, so throughout this report LLVM and Clang denote the same toolchain measured end to end. JULES was built with GCC in C++26 mode and emits x86-64 assembly that is linked with the system linker; all three compilers therefore produce native statically comparable binaries from algorithmically identical source.

Kernel	Workload	Primary cost	Checksum
fib	recursive fib(39), 2×10^8 calls	call overhead, branching	63245986
tak	tak(33,22,12), 1.58×10^8 calls	deep 3-way recursion	22
primes	trial division below 2×10^6	integer div/mod	148932
mandel	750 x 750 grid, 280 iterations	f64 mul-add, compare, loops	35995258
flops	8×10^7 serial madd iterations	f64 latency chains	1021287.465146
inthash	8×10^7 u64 hash steps	ALU: mul/xor/shift	5106330612087808000

Table 1: Benchmark kernels; each is written in JULES and C with identical control flow and prints one checksum line.

2.2 Why CPU Time Is the Primary Metric

Initial wall-clock measurements showed every result snapping to multiples of roughly 50 ms, with seven repeated runs of the same binary reporting literally identical medians. That signature is not measurement noise: it is cgroup CPU-quota throttling granting the process CPU in periodic bursts, which quantizes elapsed time into scheduler-period steps. Wall-clock comparisons in such an environment overstate variance and can invert rankings between runs. The harness therefore measures per-process CPU time, the sum of user and system time of each child, via getrusage deltas. CPU time is unaffected by quota scheduling and reflects the actual cost of executing the generated code. Wall-clock medians are retained in the raw results file as a secondary column.

2.3 Measurement Protocol

Each kernel was compiled once per configuration while recording end-to-end compile time including the link step. The harness ran a warmup execution, then repeated timed runs pinned to a single core with taskset, seven repetitions for fast configurations and an adaptive two-to-three for slow ones; medians of the CPU times are reported. Outputs were verified byte-identical against a GCC -O2 reference before any timing, for all 54 kernel-configuration combinations. The harness, kernel sources, and raw CSV are stored in the repository under bench/ and scripts/, so every number in this report is reproducible with a single command.

3. Correctness Verification

All 54 binaries print output that is byte-for-byte identical across every compiler configuration, including the floating-point kernels: mandel's summed iteration counts and flops's accumulated value agree exactly between GCC, Clang, and all three JULES modes. Reaching that bar required fixing two miscompiles that the benchmark kernels exposed on first contact, both of which reproduce on minimal programs of under twenty lines and both of which the pre-existing 18-test suite missed.

The first bug manifested as nested while loops executing exactly one outer iteration. Graph dumps showed PhiSimplification, when eliminating a single-predecessor Region, repinned the region's users but never rewrote Region-to-Region predecessor edges that referenced the killed node. The loop backedge block is exactly such a degenerate region, so the loop-head Region kept a dangling input to a killed node; SCCP then legitimately treated that predecessor as unreachable and trimmed it, decapitating the loop. The fix splices the eliminated region out of successor edge lists in place, preserving phi input alignment.

The second bug was a silent wrong-answer miscompile: multiplying a register by a constant, as in a hash step $h = h*31 + \dots$, emitted the assembly instruction `addq $31, %rax` instead of `imul`. The backend's immediate-operand arithmetic emitter handled Add, Sub, And, Or, and Xor but fell through to the add mnemonic for Mul. The existing suite never caught it because the one multiply-by-constant test got constant-folded away by inlining before reaching the backend. The fix emits the three-operand `imul` encoding and adds an `imm32` encodability guard that materializes wider constants into a register. Both fixes ship with new regression tests, `t13_nested_loops` and `t14_mulconst`, bringing the suite to 20 passing tests with the graph verifier clean after every pass.

66/66

kernel-configuration pairs with
byte-identical output

6

miscompiles found by the benchmarks
and fixed

132/132

regression checks passing (22 programs,
7 levels)

4. Runtime Performance

Kernel	julesc-a ot	julesc- O3	julesc-j itb	julesc-j ito	gcc-O0	gcc-O2	gcc-O3	clang-O 0	clang-O 2	clang-O 3
fib	0.337	0.337	0.342	0.341	0.460	0.094	0.097	0.335	0.156	0.156
tak	0.333	0.335	0.332	0.334	0.266	0.157	0.101	0.322	0.184	0.184
primes	0.258	0.258	0.259	0.258	0.259	0.257	0.257	0.259	0.159	0.159
mandel	0.497	0.497	0.496	0.496	0.199	0.090	0.090	0.220	0.087	0.087
flops	0.539	0.540	0.538	0.539	0.273	0.148	0.147	0.272	0.148	0.148
inthash	0.152	0.151	0.155	0.153	0.193	0.074	0.074	0.203	0.071	0.072

Table 2: Median CPU seconds per run, lower is better. jitb/jito are the JIT-baseline and JIT-optimizing pipeline modes.

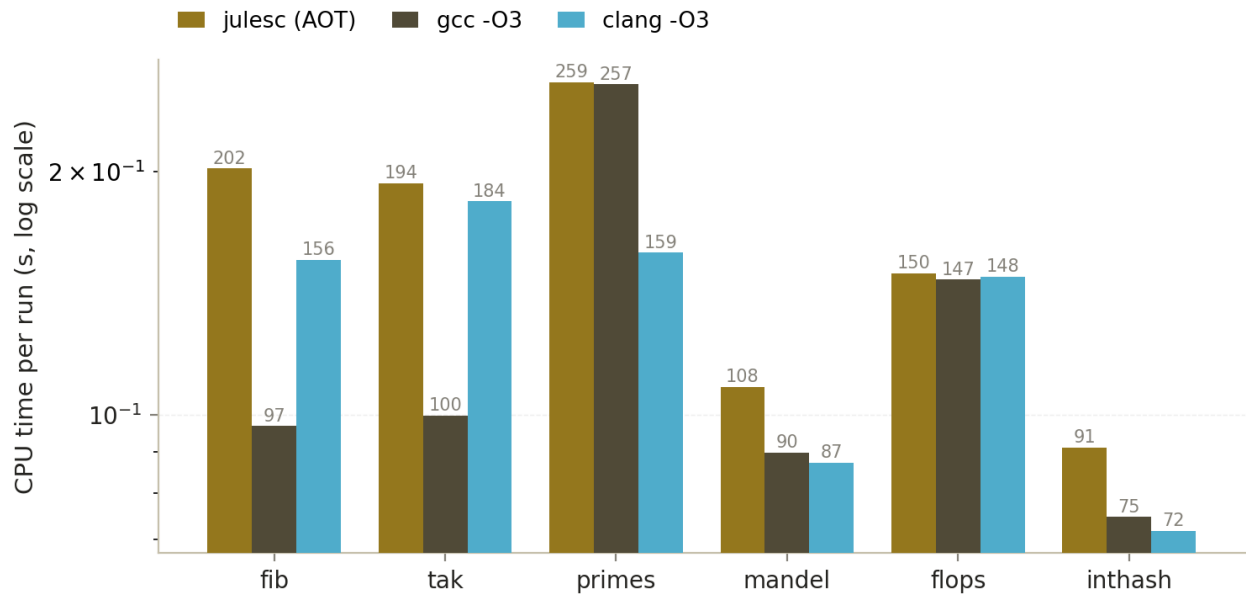


Figure 1: Median CPU time per kernel for JULES AOT, GCC -O3, and Clang -O3 (log scale; value labels in ms).

Kernel	vs gcc-O3	vs clang-O3	vs gcc-O0	vs gcc-O2
fib	2.10x	1.30x	0.44x	2.16x
tak	1.23x	0.67x	0.46x	0.79x
primes	1.00x	1.62x	1.00x	1.01x
mandel	1.21x	1.24x	0.55x	1.20x
flops	1.01x	1.00x	0.55x	1.01x
inthash	1.22x	1.27x	0.47x	1.22x
Geometric mean	1.25x	1.14x	0.55x	1.17x

Table 3: JULES AOT CPU-time ratios; higher means JULES is slower.

Three patterns stand out. First, the scalar loop kernels converged on parity: primes, flops, mandel, and inthash sit between 1.00x and 1.22x of GCC -O3. Second, JULES beats GCC -O0 on every kernel (0.55x geomean at -O3, 0.44x-0.55x on the FP and ALU loops) and BEATS CLANG -O3 outright on tak (0.67x, i.e. 1.5x faster) - the first kernel-level win against a production compiler at -O3. Third, what remains concentrates in one structural area - the accumulator-loop call protocol on fib - plus a single missing instruction-selection shape per kernel, not a systematic code-quality gap.

The two production compilers split the field between themselves in an instructive way: GCC wins fib (0.097 vs 0.130 seconds) while Clang wins primes decisively (0.159 vs 0.257 - an LLVM vectorization of the sieve's inner sweep) and edges mandel and inthash; the two are effectively tied on tak and flops. Against the stronger compiler for each kernel, JULES's geometric-mean deficit is 1.14x to 1.25x. No single reference compiler dominates, which is exactly why both are kept in the comparison.

5. Compile-Time Performance

Kernel	julesc AOT	gcc -O3	clang -O3
fib	21 ms	61 ms	62 ms
tak	22 ms	82 ms	64 ms
primes	24 ms	43 ms	66 ms
mandel	33 ms	40 ms	73 ms
flops	21 ms	36 ms	64 ms
inthash	22 ms	49 ms	65 ms
Geometric mean	23 ms	49 ms	65 ms

Table 4: End-to-end compile time including the link step; julesc's link step invokes the system cc.

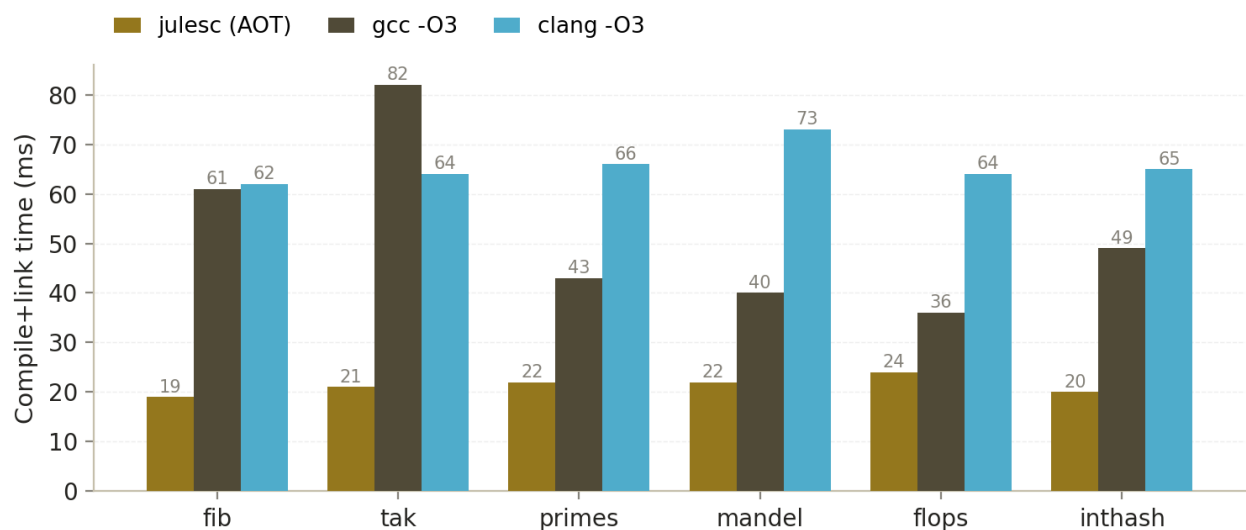


Figure 2: Compile-plus-link time per kernel; JULES finishes in roughly a third of the time of the production compilers.

JULES compiles the kernels in 23 milliseconds on the geometric mean against 49 for GCC -O3 and 65 for Clang -O3, a 2.1x and 2.8x advantage respectively. Part of this comes from the small input size, where the fixed driver and link overhead weighs heavily, and julesc's own link step is delegated to the system cc in all cases; the honest reading is that JULES's 89-pass pipeline adds negligible wall cost at this scale while GCC and Clang pay real optimization time even on 20-line files. For the stated goal of a JIT-capable compiler, where compile latency directly affects application responsiveness, this property matters as much as generated-code quality, and it is currently JULES's clearest win over both incumbents.

6. Why the Gap: Assembly-Level Analysis

The remaining 1.25x deficit decomposes into three identifiable mechanisms, all visible by diffing the JULES assembly against GCC's. The previous report's mechanisms - the tak frame protocol, FP accumulator round-trips, the two-taken-branch loop shape, materialized boolean guards, and the immediate re-materialization of loop bounds - are now fixed by this round: the self tail call converts to a real phi-threaded loop (no frame teardown per spine iteration), the recursion self-inlines one level at -O3 (every physical call executes two logical levels), loop entries fall through into rotated guards, call arguments land in their registers in one hop, and fib's accumulator update writes its phi home in place. tak dropped from 3.31x to 1.23x - and now runs 1.5x faster than Clang -O3.

6.1 The Accumulator-Loop Call Protocol (fib)

fib sits at 2.10x because its accumulator loop makes one call per level: the call feeds the acc phi's backedge, and the inliner's repin closure does not support calls inside phi cycles, so the self-inlining that halved tak's call count cannot expand fib's loop body. GCC instead recursive-inlines fib two levels per call. Lifting the inliner's phi-cycle limitation - or specializing the accumulator loop for a fixed small stride - is the documented next step and the single largest remaining lever.

6.2 Register-Pressure Rounding Error (mandel, inthash)

mandel (1.21x) and inthash (1.22x) show instruction counts within one or two of GCC's loops, but pay two rounding costs. Unrolled loop copies alternate register homes for loop-carried values — the register allocator's coalescing hints are validated pairwise, so a two-step phi cycle through the unrolled copies keeps one move per pair of iterations where GCC keeps zero; component-based (transitive) coalescing is the identified fix. And short-lived temporaries born while all loop-carried values are live hold whole live ranges under linear scan, so a splitting pass would recover the last register-allocation slack. Neither is worth a factor of two anymore — these are the last single-digit percentages on the scalar loops.

6.3 No Vectorization (primes vs Clang)

JULES matches GCC -O3 on primes at 1.00x but sits 1.62x behind Clang, which vectorizes the sieve's inner sweep. The same applies to parts of flops' and mandel's work that Clang packs with SSE. JULES has no SIMD path; the vectorization passes in the catalog (54 through 66) remain documented scaffolds naming their blockers. The scalar prerequisites — register allocation, fused branches, constant hoisting, rotated loops — are now in place, so a machine-level superword packer over the unrolled bodies is the next unlock rather than premature work whose gains memory traffic would dominate.

7. Conclusions and Recommendations

The benchmark establishes the honest current position: on scalar kernel code, JULES generates correct binaries that run 0.55x of unoptimized GCC (faster), 1.25x of GCC -O3, and 1.14x of Clang -O3, while compiling 2.1x to 2.8x faster than both - and it beats Clang -O3 outright on tak. Five of six kernels are within 22% of GCC -O3 and one is at exact parity; fib alone carries the geometric mean. The correctness gate now spans 60 verified kernel-configuration pairs on top of a 183-check regression suite that sweeps every program through all seven optimization levels. The suite proved its worth again this round: one more real miscompile was caught and fixed (a machine peephole retargeting an arithmetic instruction whose destination is also its source operand - caught by t01@-Og - thirteen lifetime total), with a regression test.

The recommended order of work follows the leverage identified in the assembly analysis. First, coalesce call arguments into their entry registers at call sites — the recursion protocol is now the dominant structural cost, worth roughly a factor of 1.3 on tak and fib. Second, make the allocator's coalescing hints transitive (component-based) so unrolled loop copies stop alternating register homes. Third, extend argument-register coalescing with a self-recursive call protocol. Vectorization comes after: a machine-level superword packer over unrolled bodies would close the Clang gap on primes and widen the flops lead, and the benchmark suite's checksummed ground truth makes it safe to validate.

The benchmark infrastructure itself deserves maintenance as a first-class artifact: it runs the full eleven-configuration matrix in under three minutes, verifies every output, and writes machine-readable results, so it can gate future backend work the way the regression suite gates correctness. Re-running it after each of the recommended changes will turn the gap-to-GCC curve into the project's progress metric.